PRODUCT REQUIREMENTS DOCUMENT

Week 1: Environment Setup & First Transcription

Caption Generator with Translation — Desktop App

Project Phase 1 of 8 | Estimated Duration: 7 Days

1. Document Overview

Document Title	Week 1 PRD — Environment Setup & First Transcription
Project	Caption Generator with Translation (Desktop App)
Phase	Week 1 of 8
Prepared For	Antigravity (Autonomous Coding Agent)
Tech Stack	Python, OpenAI Whisper (local), HuggingFace Transformers, PyTorch CUDA
Target Machine	Windows PC with NVIDIA GTX 1650 (4GB VRAM), CUDA-capable
Status	Ready for Implementation
Dependencies	None — this is the first phase

2. Objective

The goal of Week 1 is to establish a fully working local Python environment that can:

- Transcribe a given audio file (.mp3 / .wav) to text using OpenAI Whisper running locally on the GPU.
- Translate a sentence from English to Tamil using a HuggingFace Helsinki-NLP MarianMT model running locally.
- Confirm that all ML inference is running on the GPU (GTX 1650) and not on the CPU.
- Produce a clean, well-structured project folder that the rest of the project will build upon.

By the end of Week 1, running a single Python script should print a transcription of a test audio file followed by its Tamil translation — entirely offline, no API calls.

3. Background & Context

This project is a college NLP assignment. The full application is a desktop caption generator with live mic and file-based transcription, translation between English and Tamil, and an always-on-top overlay window similar to Discord's overlay.

The faculty has explicitly disallowed the use of cloud APIs (such as OpenAI's Whisper API or Google Translate API). All models must run locally on the developer's machine.

> AGENT NOTE: Never use API keys, cloud endpoints, or any external service calls for ML tasks. All model inference must be local. The only network activity allowed in this phase is pip/npm installs and HuggingFace model downloads on first run.

4. Scope

4.1 In Scope

- Python environment setup (Python 3.10+, pip, virtual environment)
- PyTorch installation with CUDA 11.8 support for GTX 1650
- OpenAI Whisper (openai-whisper pip package) installation and configuration
- HuggingFace Transformers installation
- Helsinki-NLP opus-mt-en-ta and opus-mt-ta-en model download and test
- A working transcribe.py script that accepts an audio file and returns text
- A working translate.py script that translates a given string EN<->TA
- A combined test script: test_pipeline.py
- GitHub repository initialization with correct folder structure
- requirements.txt with pinned versions

4.2 Out of Scope

- Flask server — that is Week 2
- Electron/React UI — that is Week 3 onwards
- Live microphone input — that is Week 4
- Overlay window — that is Week 4
- Any cloud API calls

5. Required Project Folder Structure

The agent must create the following folder structure. This is the canonical structure the entire project will use.

```
caption-app/
├── backend/
│   ├── transcriber.py          # Whisper logic
│   ├── translator.py           # HuggingFace translation logic
```

```
│   ├── audio_capture.py        # (empty placeholder for Week 4)
│   ├── app.py                  # (empty placeholder for Week 2)
│   └── test_pipeline.py        # Week 1 integration test script
├── electron/
│   ├── main.js                 # (empty placeholder)
│   ├── index.html              # (empty placeholder)
│   ├── overlay.html            # (empty placeholder)
│   └── renderer.js             # (empty placeholder)
├── assets/
│   └── test_audio.mp3          # A short test audio file (agent must source or
generate)
├── .gitignore
├── requirements.txt
├── README.md
└── package.json               # (empty placeholder, init with npm init -y)
```

> AGENT NOTE: Create all placeholder files as empty files. Do not leave any folders missing. The electron/ folder and package.json should exist but be empty stubs — they will be filled in later phases.

6. Technical Specifications

6.1 Python Environment

Requirement	Specification
Python Version	3.10 or 3.11 (not 3.12 — some packages have issues)
Virtual Environment	venv — must be created at caption-app/venv/
Package Manager	pip (with --break-system-packages if needed)
OS Target	Windows 10/11 (primary), Linux compatible

6.2 PyTorch Installation

CRITICAL: PyTorch must be installed with CUDA 11.8 support. The GTX 1650 supports CUDA 11.x. Do NOT install the CPU-only version.

Use the following exact pip command:

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

After installation, verify CUDA is available by running:

```
python -c "import torch; print(torch.cuda.is_available());
print(torch.cuda.get_device_name(0))"
```

> AGENT NOTE: The output must print True and the GPU name (e.g. NVIDIA GeForce GTX 1650). If it prints False, PyTorch was installed without CUDA and must be reinstalled. Do not proceed until this check passes.

6.3 Whisper Model

Parameter	Value
Package	openai-whisper (pip install openai-whisper)
Model Size	small (this is mandatory — see rationale below)
VRAM Usage	~2GB (GTX 1650 has 4GB — safe headroom)
Device	cuda (must not fall back to cpu)
Language Support	English and Tamil both supported natively
Auto-detect language	Yes — Whisper can auto-detect EN or TA

Model size rationale: The large model requires ~10GB VRAM (exceeds GTX 1650). The medium model requires ~5GB (borderline risky). The small model requires ~2GB and provides good accuracy for both English and Tamil — it is the correct choice for this hardware.

6.4 Translation Models

Direction	Model ID	Notes
English → Tamil	Helsinki-NLP/opus-mt-en-ta	Download ~300MB on first run
Tamil → English	Helsinki-NLP/opus-mt-ta-en	Download ~300MB on first run

Both models use the MarianMT architecture from HuggingFace Transformers. They run on CPU by default but can be moved to GPU. For this phase, CPU inference is acceptable for translation (it is fast enough for short sentences).

7. Implementation Specifications

7.1 transcriber.py

This module wraps OpenAI Whisper. It must expose a single function: transcribe(audio_path, language=None).

Function Signature:

```
def transcribe(audio_path: str, language: str = None) -> dict:
    """
    Transcribe an audio file using Whisper small model on GPU.

    Args:
        audio_path (str): Absolute or relative path to .mp3 or .wav file.
        language (str): Optional. 'en' or 'ta'. If None, Whisper auto-detects.

    Returns:
        dict: {
            'text': str,           # Full transcription
            'language': str,       # Detected/used language code
            'segments': list       # List of timed segment dicts
        }
    """
```

Implementation Requirements:

- Load model ONCE at module level (not inside the function) to avoid reloading on every call.
- Model must be loaded with device='cuda'. Raise a clear RuntimeError if CUDA is not available.
- Use model.transcribe(audio_path, language=language, fp16=True) — fp16=True for GPU efficiency.
- Log to console: model loading time, transcription time, detected language.
- Handle FileNotFoundError gracefully with a helpful error message.

Expected Console Output on Success:

```
[Whisper] Loading model 'small' on cuda...
[Whisper] Model loaded in 3.2s
[Whisper] Transcribing: assets/test_audio.mp3
[Whisper] Done in 4.1s | Detected language: en
[Whisper] Transcript: Hello, this is a test audio file.
```

7.2 translator.py

This module wraps HuggingFace MarianMT. It must expose a single function: translate(text, direction).

Function Signature:

```
def translate(text: str, direction: str = 'en-ta') -> str:
    """
    Translate text between English and Tamil.
```

```
    Args:
        text (str): Input text to translate.
        direction (str): 'en-ta' for English to Tamil, 'ta-en' for Tamil to
English.

    Returns:
        str: Translated text.
    """
```

Implementation Requirements:

- Load BOTH models and tokenizers at module level (not inside the function).
- Models: Helsinki-NLP/opus-mt-en-ta and Helsinki-NLP/opus-mt-ta-en.
- Use AutoTokenizer and MarianMTModel from transformers.
- Tokenize with return_tensors='pt', padding=True, truncation=True, max_length=512.
- Raise ValueError if direction is not 'en-ta' or 'ta-en'.
- Log model loading and translation time to console.
- Return the decoded translated string (skip_special_tokens=True).

Expected Console Output on Success:

```
[Translator] Loading en-ta model (Helsinki-NLP/opus-mt-en-ta)...
[Translator] Loading ta-en model (Helsinki-NLP/opus-mt-ta-en)...
[Translator] Models loaded.
[Translator] Translating en->ta: 'Hello, this is a test.'
[Translator] Result: 'வணக்கம், இது ஒரு சோதனை.'
[Translator] Done in 0.8s
```

7.3 test_pipeline.py

This is the Week 1 integration test. Running this script must validate the entire pipeline end to end.

Script Behavior:

1. Load Whisper model and confirm it is on CUDA.
2. Load both Helsinki-NLP translation models.
3. Transcribe assets/test_audio.mp3 using Whisper.
4. Print the transcription to console.
5. Translate the transcription from English to Tamil.
6. Print the Tamil translation to console.
7. Print a PASS/FAIL summary at the end.

Expected Terminal Output (example):

```
============================================
  WEEK 1 PIPELINE TEST
============================================
[Whisper] Loading model 'small' on cuda...
[Whisper] Model loaded in 3.2s
[Translator] Loading models...
[Translator] Models loaded.

--------------------------------------------
[TEST 1] CUDA Available      : PASS (NVIDIA GeForce GTX 1650)
[TEST 2] Whisper Transcribe  : PASS
         Result: 'Hello this is a test audio file for the caption app.'
[TEST 3] EN->TA Translation  : PASS
         Result: 'வணக்கம் இது ஒரு சோதனை ஆடியோ கோப்பு.'
[TEST 4] TA->EN Translation  : PASS
         Result: 'Hello this is a test audio file.'
--------------------------------------------
ALL TESTS PASSED. Week 1 complete.
============================================
```

AGENT NOTE: If any test fails, print a descriptive error message and exit with a non-zero code (sys.exit(1)). Do not silently swallow exceptions. The output above is the target — the agent should aim to match it exactly.

8. Test Audio File

A test audio file is required at assets/test_audio.mp3. The agent must create or source a short audio file for testing. Options in order of preference:

- Option A (Preferred): Use the gTTS (Google Text-to-Speech) library to programmatically generate a test audio file from a known string. This is fully offline after pip install gtts and produces a deterministic result.

```
# generate_test_audio.py
from gtts import gTTS
tts = gTTS('Hello, this is a test audio file for the caption app.', lang='en')
tts.save('assets/test_audio.mp3')
print('Test audio saved to assets/test_audio.mp3')
```

- Option B (Fallback): Download any short public domain English speech clip and place it at assets/test_audio.mp3.

> AGENT NOTE: The test audio must be English speech — clear, single speaker, no background music. The transcript must be a complete English sentence so the translation step has meaningful input.

9. requirements.txt

The agent must create a requirements.txt in the project root with the following packages (use >= version constraints, not pinned, for flexibility):

```
# ML / AI
openai-whisper>=20231117
transformers>=4.35.0
sentencepiece>=0.1.99
sacremoses>=0.0.53

# Audio
sounddevice>=0.4.6
numpy>=1.24.0
ffmpeg-python>=0.2.0

# TTS (for test audio generation only)
gTTS>=2.3.2

# Web server (placeholder for Week 2)
flask>=3.0.0
flask-socketio>=5.3.6

# NOTE: PyTorch must be installed separately with CUDA:
# pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

> AGENT NOTE: PyTorch is intentionally NOT in requirements.txt because it requires a special index URL for CUDA support. Document this clearly in README.md under the Installation section.

10. .gitignore

Create a .gitignore in the project root with at minimum the following entries:

```
# Python
venv/
__pycache__/
*.pyc
*.pyo
.env
```

```
# HuggingFace model cache (large files)
.cache/
models/

# Node
node_modules/
dist/

# OS
.DS_Store
Thumbs.db

# Test artifacts
assets/*.mp3
assets/*.wav
```

11. README.md

Create a README.md with the following sections:

- Project title and one-line description
- Tech stack table (same as in the project roadmap)
- Installation instructions — step by step, starting from scratch
  - Clone the repo
  - Create and activate venv
  - Install PyTorch with CUDA (special command)
  - pip install -r requirements.txt
  - Run python backend/test_pipeline.py
- Section headings for Week 2, Week 3 etc. (empty for now — will be filled later)

12. Acceptance Criteria

Week 1 is complete ONLY when ALL of the following are true:

#	Acceptance Criterion	How to Verify
AC-01	PyTorch detects GTX 1650 via CUDA	torch.cuda.is_available() returns True
AC-02	Whisper small model loads on GPU	No CPU fallback warning in logs

AC-03	Audio file transcription works	test_pipeline.py prints a non-empty transcript
AC-04	English to Tamil translation works	Output contains Tamil Unicode characters
AC-05	Tamil to English translation works	Output is readable English text
AC-06	test_pipeline.py prints ALL TESTS PASSED	No FAIL lines in output
AC-07	Folder structure matches Section 5	All files and folders present
AC-08	requirements.txt is present	pip install -r requirements.txt runs without error
AC-09	GitHub repo is initialized	git log shows at least one commit
AC-10	README.md has installation steps	A new developer can follow them from scratch

13. Known Risks & Mitigations

Risk	Mitigation
CUDA not detected after PyTorch install	Reinstall with the correct --index-url cu118 command. Verify NVIDIA drivers are up to date.
Whisper falls back to CPU (too slow)	Explicitly pass device='cuda' when loading model. Check CUDA availability first.
HuggingFace model download fails	Check internet connection. Models are cached after first download at ~/.cache/huggingface/
ffmpeg not found (Whisper dependency)	Install ffmpeg separately: winget install ffmpeg (Windows) or apt install ffmpeg (Linux)
Tamil characters not displaying in terminal	This is a display issue only — the data is correct. Check Windows Terminal Unicode support.
GTX 1650 VRAM overflow	Ensure no other GPU-heavy applications are running. Use fp16=True in Whisper.

14. Handoff Notes to Agent

READ THIS SECTION CAREFULLY BEFORE STARTING IMPLEMENTATION.

- Do not ask clarifying questions — all decisions have been made in this document. Implement exactly as specified.

- If a package version conflict arises, resolve it using the latest compatible version and document the change in requirements.txt as a comment.
- All console output must use the exact prefixes shown (e.g. [Whisper], [Translator], [TEST N]) — the frontend will parse these in later phases.
- Do not import or use any OpenAI API client libraries. The openai-whisper package is a local model package, NOT an API client.
- Model loading must happen at module level — not inside functions. This is critical for performance in later phases when Flask will import these modules.
- On completion, run test_pipeline.py one final time and confirm ALL TESTS PASSED is printed before committing.
- Git commit message for the final commit: feat(week1): environment setup and pipeline test passing

---

End of Week 1 PRD

Caption Generator with Translation Project